

Parcours client — fiche de soutenance

Ce que tu montres sur mobile, et ce qu'il faut pouvoir en dire. Établi par lecture du code, pas de mémoire : chaque chiffre cité ici vient d'une ligne précise.

État du code : branche `migration-sdk57` de `coach-mobile-app`. Trois comportements de l'écran de séance y sont récents — RPE retiré, chrono de repos automatique, bouton de lancement sur les exercices en temps.

Le fil rouge de la démonstration : on est en salle, debout, entre deux séries, avec les mains moites. Chaque décision d'interface découle de ça. C'est l'argument qui relie les quatre blocs ci-dessous ; s'il ne reste qu'une phrase au jury, c'est celle-là.

Sommaire

1. La messagerie
 2. La complétion d'une séance
 3. Le suivi quotidien
 4. Les repas
 5. Ce qui remonte au coach
 6. Questions probables du jury
-

1. La messagerie

Ce que tu montres

La liste des coaches, une conversation, l'envoi d'un message, et une proposition de rendez-vous reçue du coach qu'on accepte.

Ce qui se passe techniquement

Il n'existe pas d'endpoint « liste de conversations ». La colonne de gauche est construite par fusion de deux sources : les coaches rattachés au profil client, et `GET /messages/conversations` — dédoublés par identifiant. Si le second appel échoue, on retombe silencieusement sur les seuls coaches rattachés.

Le fil est rafraîchi par sondage toutes les 3 000 ms — `MessagesScreen.js:182-190`. Il n'y a **pas de WebSocket** : c'est une limite assumée, et il vaut mieux l'annoncer que la laisser découvrir.

L'envoi n'est pas optimiste. On attend la réponse du serveur, puis on recharge le fil entier. Le champ n'est vidé qu'après succès : si l'envoi échoue, le texte saisi est conservé.

Le marquage comme lu part dans deux cas : à l'ouverture d'une conversation qui a des non-lus, et à chaque sondage s'il y a un message non lu venant de l'autre partie **et** que l'onglet est visible. Ce dernier garde-fou évite de marquer comme lus des messages que personne ne regarde.

Trois types de message, rendus différemment :

Type	Rendu
CHAT	bulle classique, alignée à droite si c'est moi
TIP	bulle centrée pleine largeur, icône info — un conseil du coach
APPOINTMENT_PROPOSAL	carte de rendez-vous avec date, durée, lieu et statut

Le rendez-vous dans la conversation est le point le plus transverse du projet : il touche la messagerie, l'agenda et les notifications d'un coup. Les deux parties peuvent proposer ; **seul le client peut accepter ou refuser**, et seulement une proposition venue du coach. Accepter appelle `PUT /appointments/{id}/confirm`, refuser appelle `PUT /appointments/{id}/cancel?scope=single`.

Statuts : `PROPOSED` (« En attente de confirmation »), `CONFIRMED`, `CANCELLED`.

La cloche de notifications sonde `GET /notifications/unread-count` toutes les **30 000 ms**. La liste n'est chargée qu'à l'ouverture. Un clic sur une notification de rendez-vous bascule sur l'onglet Agenda.

La phrase à dire

*C'est la réponse directe au problème posé en ouverture : le coach échange avec ses clientes depuis l'application. Et la proposition de rendez-vous part **de la conversation**, sans changer d'écran — c'est là qu'on en parle, c'est là qu'on la fait.*

⚠ À ne pas dire

Ne parle pas de « temps réel ». C'est du sondage à 3 secondes, et un jury technique fera la différence. Dis « rafraîchissement toutes les trois secondes », et enchaîne sur le fait qu'un WebSocket serait la suite logique.

2. La complétion d'une séance

Ce que tu montres

Démarrer la séance, valider une série, le chrono de repos qui part tout seul, un exercice de renforcement saisi en temps, l'échauffement lancé au bouton, puis la validation finale.

Les trois chronomètres, à ne pas confondre

C'est le point où l'on peut s'embrouiller à l'oral. Ils sont bien distincts :

	Ce qu'il mesure	Comment il part
Chrono de séance	la durée totale, du premier au dernier exercice	bouton « Démarrer la séance », pause et reprise possibles
Chrono de repos	le temps entre deux séries	tout seul , à la validation d'une série
Chrono d'exercice	la durée d'un échauffement, d'un étirement, d'un cardio	bouton « Lancer » sur l'exercice

Le chrono de séance ne compte pas le temps en pause : il accumule les segments actifs (`accumulatedRef` + le segment courant). C'est cette valeur qui part au serveur à la validation, en secondes.

Les deux autres partagent la même barre de décompte, qui prend le libellé de ce qu'elle mesure — « TEMPS DE REPOS » ou le nom de l'exercice.

La saisie d'une série

Deux gestes, pas plus : les valeurs prévues par le coach sont pré-remplies, le client ne corrige que ce qui a changé et coche. Trois champs seraient déjà trop en salle.

Chaque validation part immédiatement au serveur — `PUT /set-completions` — et **le chrono de repos démarre dans la foulée**, avec la durée prévue par le coach, ou 90 secondes par défaut si l'exercice n'en précise aucune.

Le renforcement, saisi en temps

Un exercice de catégorie `RENFORCEMENT` n'affiche **ni répétitions ni charge : un temps tenu**. Ce n'est pas cosmétique, et c'est un excellent point à faire valoir :

*Un gainage de 45 secondes enregistré dans le champ « répétitions » serait compté comme **45 répétitions** dans le calcul du volume. Le volume vaut poids × répétitions : la statistique serait faussée d'un facteur soixante. Les deux champs sont donc exclusifs, côté saisie **et** côté base.*

C'est aussi ce qui explique le travail de reclassement fait en production : 225 exercices de gainage et de chaise ont été repassés en `RENFORCEMENT`.

La validation finale

Le bouton reste **désactivé tant que toutes les séries ne sont pas cochées** — libellé « Complétez toutes les séries », qui devient « Valider la séance ». Après confirmation, `PUT /sessions/{id}/validate` envoie `{ durationSeconds }`, la séance passe en `DONE` et `completedByClient` à vrai.

⚠ Le piège de la démonstration

La séance de démonstration a **10 séries au total**. Tu ne pourras pas toutes les cocher en direct sans y passer deux minutes. Deux options : coche-en deux ou trois et explique la règle sans aller jusqu'au bout, ou pré-coche tout sauf une avant d'entrer dans la salle.

3. Le suivi quotidien

Ce que tu montres

La frise de dates, la saisie du poids et de l'hydratation, et l'indicateur « Enregistré » qui apparaît tout seul.

Ce qui se passe techniquement

Il n'y a pas de bouton Enregistrer. La sauvegarde part **900 ms après la dernière frappe** —

`DashboardScreen.js:347`. C'est le point à mettre en avant :

Une donnée saisie est une donnée conservée. Personne ne pense à appuyer sur « Enregistrer » entre deux séries.

Un indicateur remplace le bouton : « Enregistrement... », « Enregistré », ou « Enregistrement impossible — vos données restent affichées ».

Huit champs : heure de coucher, heure de réveil, eau (L), poids (kg), heure de séance, durée de séance (min), notes — et le **sommeil, qui n'est pas saisi mais calculé** : réveil du jour moins coucher de la veille, retenu seulement si l'écart est entre 0 et 24 h. Le coucher de la veille est récupéré par un second appel.

Côté serveur, c'est un `upsert` sur la clé composite `clientId + date`. Détail qui a son importance : en mise à jour, une valeur nulle est ignorée plutôt qu'écrite — **saisir le poids n'efface pas l'hydratation saisie ce matin**.

⚠ Deux limites que je te conseille d'assumer

La saisie est perdue si on quitte l'écran dans les 900 ms. Le minuteur est annulé au démontage, sans envoi de secours. C'est étroit mais réel. Si on te le demande : la correction est un envoi forcé au démontage, elle est identifiée.

Il n'y a aucun objectif ni barre de progression côté client. Le coach définit bien un objectif d'hydratation et un objectif calorique au niveau du programme — mais l'application mobile ne les relit jamais. Ne promets donc pas de « progression vers l'objectif » : montre des totaux bruts, et présente les objectifs comme une évolution identifiée.

4. Les repas

Ce que tu montres

Choisir « Déjeuner », chercher un aliment, ajuster la quantité, valider — et voir les calories du jour s'incrémenter.

Ce qui se passe techniquement

Quatre types : petit-déjeuner, déjeuner, dîner, collation. Un seul actif à la fois.

La base d'aliments est la table Ciqua, hébergée en local dans notre PostgreSQL — ce n'est pas un appel à une API tierce. C'est un point fort : pas de dépendance externe, pas de latence réseau, pas de clé d'API à gérer. La recherche est un `ILIKE`, avec les correspondances par le début en tête de liste.

Anti-rebond de 400 ms, minimum 2 caractères, 8 résultats affichés. On ne lance pas une requête à chaque lettre.

Le calcul des macros est proportionnel : valeur pour 100 g ÷ 100 × quantité. La quantité vaut 100 g par défaut. Calories arrondies à l'entier, macros à 0,1 g près.

L'ajout est optimiste — contrairement à l'envoi de message. L'aliment apparaît immédiatement, puis est remplacé par l'objet du serveur pour récupérer son identifiant. En cas d'échec il reste affiché, marqué non enregistré.

Une **saisie manuelle** est prévue pour ce qui n'est pas dans la base : description, calories, macros. Le bouton reste désactivé tant que description et calories sont vides.

⚠ Ce qu'il ne faut pas promettre

Il n'y a pas de photo de repas dans l'application mobile. Le serveur sait pourtant la recevoir — la route accepte un fichier et le stocke. C'est branché côté serveur, pas côté client. Si tu veux en parler, dis-le exactement comme ça : c'est une évolution dont la moitié est déjà faite, et ça se défend mieux qu'un silence.

Le **calculateur de calories** (Mifflin-St Jeor, facteurs 1,2 à 1,9) est réservé au coach, et la route serveur elle-même exige le rôle `COACH`. Ne le cherche pas côté client.

5. Ce qui remonte au coach

C'est la boucle à fermer en fin de démonstration.

Ce que la cliente fait	Ce que le coach voit
Valide ses séries	séance à <code>DONE</code> , charges et répétitions réelles, durée totale
Saisit poids, eau, sommeil, durée de séance	fiche client → suivi du jour
Ajoute un repas	le total calorique du jour est recalculé côté serveur et écrit dans la statistique quotidienne

Le point technique élégant, et qui montre une vraie décision d'architecture :

Chaque ajout ou suppression de repas déclenche un recalcul du total calorique du jour côté serveur. Le total n'est jamais calculé par le client : la donnée reste juste même si deux appareils saisissent en même temps.

⚠ **La durée de séance est enregistrée mais n'est affichée nulle part dans le dashboard coach.** Elle arrive en base, elle est renvoyée par l'API, aucun écran ne la montre. Ne l'annonce pas comme visible.

Attention aussi à ne pas confondre deux données proches : le « temps d'entraînement » que le coach voit dans les moyennes sur 14 jours vient de la **saisie déclarative** du client, pas du chronomètre de la séance.

6. Questions probables du jury

« **C'est du temps réel ?** » Non, du sondage : 3 secondes pour les messages, 30 pour les notifications. Un WebSocket serait la suite logique ; le sondage a suffi pour la charge visée et a coûté une journée de moins.

« **Pourquoi pas de bouton Enregistrer ?** » Parce que la saisie a lieu en salle, entre deux séries. Un bouton oublié, c'est une donnée perdue. On enregistre 900 ms après la dernière frappe, avec un indicateur d'état.

« **Comment gérez-vous les exercices au poids du corps ?** » Champ distinct en base, saisie distincte à l'écran, et exclusion mutuelle avec les répétitions — sinon 45 secondes de gainage deviennent 45 répétitions dans le volume.

« **Et si le réseau tombe pendant la séance ?** » Chaque série validée part immédiatement, donc ce qui est fait est enregistré. En revanche il n'y a pas de file d'attente hors ligne : une validation qui échoue n'est pas rejouée. C'est une limite connue, et la première évolution que je ferais.

« **Les notifications fonctionnent-elles ?** » Les notifications locales oui — le chrono de repos en programme une. Les notifications **push** ne fonctionnent pas dans Expo Go depuis le SDK 53 : il faudrait un *development build*. C'est un choix d'outillage, pas une limite du code.

« **Pourquoi une base d'aliments locale ?** » Ciquel est la table de référence de l'ANSES. L'héberger évite une dépendance externe, une clé d'API et la latence — et la recherche reste sous la milliseconde côté base.